Ficha de Declaração de Uso de IA

Modelo padronizado conforme o Art. 7º do Guia para o Uso Responsável de Inteligência Artificial na FECAP (versão 2.0, de 29 de julho de 2026).

IDENTIFICAÇÃO

NOME COMPLETO	RA
Daniel Sobral Quio	26028696
João Pedro da Silva Souza	26029757
Murilo Verissimo Pereira	26029666
Gisele Ribeiro dos Santos	26029631

Prompts principais utilizados

07/11/2026, 22:47 Projeto C# quiz Arcor com Windows Forms - Claude

Projeto C# quiz Arcor com Windows Forms | Compartilhar

Formato de Entrega: Projeto em C# na pasta SRC do Github.
Objetivo: Implementar e uma chamada a um servidor externo utilizando JSON.

Tenho um projeto em C# para a criação de um quiz de produtos da empresa Arcor.
Tenho que desenvolve-lo com windows forms.

No projeto há a pasta src com as subpastas
entrega 1

- pasta backend (aqui há um projeto .cs)
- pasta frontend (vazio)

entrega 2

- pasta backend (vazio)
- pasta frontend (projeto.winforms)

Mostrar menos

Memória atualizada quiz-arcor-csharp.md

Pelo que você descreveu, a chamada ao servidor externo com JSON fica no projeto WinForms da **entrega 2/frontend**. A ideia é separar em três partes: um modelo, um serviço HTTP e o formulário.

Estrutura sugerida

```
src/
├── entrega 1/
│   ├── backend/   (seu projeto .cs atual)
│   └── frontend/
└── entrega 2/
    ├── backend/
    └── frontend/
        └── QuizArcor/
            ├── Models/Pergunta.cs
            ├── Services/QuizApiService.cs
            ├── Form1.cs
            └── Program.cs
```

1. Modelo (Models/Pergunta.cs)

07/11/2026, 22:47 Projeto C# quiz Arcor com Windows Forms - Claude

```
using System.Text.Json.Serialization;

namespace QuizArcor.Models;

public class Pergunta
{
    [JsonPropertyName("id")]
    public int Id { get; set; }

    [JsonPropertyName("enunciado")]
    public string Enunciado { get; set; } = "";

    [JsonPropertyName("opcoes")]
    public List<string> Opcoes { get; set; } = new();

    [JsonPropertyName("respostaCorreta")]
    public int RespostaCorreta { get; set; }
}
```

2. Serviço que chama o servidor (Services/QuizApiService.cs)

csharp

```
        double pctFacil = d.QuestoesFaceis > 0 ? d.AcertosFaceis * 100.0 / d.Qu
        double pctMedio = d.QuestoesMedias > 0 ? d.AcertosMedias * 100.0 / d.Qu
        double pctDificil = d.QuestoesDificeis > 0 ? d.AcertosDificeis * 100.0
        double pctGeral = acertos * 100.0 / total;

        int penalidade = d.DicasUsadas * 5;
        int pontuacao = Math.Max(0,
            d.AcertosFaceis * 10 + d.AcertosMedias * 20 + d.AcertosDificeis * 3
        int pontuacaoMax = d.QuestoesFaceis * 10 + d.QuestoesMedias * 20 + d.Qu

        double tempoMedio = d.TempoPartida * 1.0 / total;

        string classificacao = pctGeral >= 90 ? "Mestre das Marcas"
                             : pctGeral >= 70 ? "Conhecedor de Marcas"
                             : pctGeral >= 50 ? "Aprendiz"
                             : "Iniciante";

        string ritmo = tempoMedio <= 10 ? "Rápido"
                     : tempoMedio <= 20 ? "Normal"
                     : "Pausado";

        string melhorNivel = "Nenhum";
        if (d.QuestoesDificeis > 0
            && (d.QuestoesMedias == 0 || pctDificil >= pctMedio)
            && (d.QuestoesFaceis == 0 || pctDificil >= pctFacil))
            melhorNivel = "Difícil";
        else if (d.QuestoesMedias > 0
            && (d.QuestoesFaceis == 0 || pctMedio >= pctFacil))
            melhorNivel = "Médio";
        else if (d.QuestoesFaceis > 0)
            melhorNivel = "Fácil";

        return new ResultadoPartida
        {
            Nickname = d.Nickname,
            NomeFaixa = Faixas[d.FaixaEtaria - 1],
            TotalQuestoes = total,
            TotalAcertos = acertos,
            TotalErros = total - acertos,
            PercentualGeral = pctGeral,
            PctFacil = pctFacil,
            PctMedio = pctMedio,
            PctDificil = pctDificil,
            Pontuacao = pontuacao,
            PontuacaoMaxima = pontuacaoMax,
            AproveitamentoPontuacao = pontuacaoMax > 0 ? pontuacao * 100.0 / po
```



```
            DicasUsadas = d.DicasUsadas,
            PenalidadeDicas = penalidade,
            TempoPartida = d.TempoPartida,
            TempoMedioPorQuestao = tempoMedio,
            Ritmo = ritmo,
            MelhorNivel = melhorNivel,
            Classificacao = classificacao
        };
    }
}
```

Pequena melhoria em relação ao console: usei `Math.Max`, operador ternário e proteção contra divisão por zero em `pontuacaoMax`, e corrigi os nomes de variável com erro de digitação (`pctFaacil`, `acertosnMedias`).

Services/QuizApiService.cs (a chamada JSON)

csharp

- GitHub: adicione um `.gitignore` do Visual Studio (`bin/`, `obj/`, `.vs/`) para não subir arquivos gerados.

Qual é o servidor externo que você vai usar: já existe uma API definida pelo professor/empresa, ou você precisa criar uma?



o conteúdo do projeto inicial na entrega 1:

Adicionado à memória `quiz-arcor-csharp.md`

Obrigado, agora ficou claro: a entrega 1 é o **resumo de uma partida** (entrada, validação, cálculo e saída) em console. Na entrega 2, o caminho natural é:

1. Mover a lógica (validação e cálculo) para classes reutilizáveis, sem `Console` e sem `Environment.Exit`.
2. Trocar o `Console.ReadLine` por **controles do WinForms**.
3. Adicionar a **chamada JSON ao servidor externo**: enviar o resultado da partida (POST).

Estrutura em `src/entrega 2/frontend/QuizArcor/`

```
Models/DadosPartida.cs
Models/ResultadoPartida.cs
Services/CalculadoraPartida.cs
Services/QuizApiService.cs
Form1.cs
```

Models/DadosPartida.cs (entradas)

csharp

07/11/2026, 22:47 Projeto C# quiz Arcor com Windows Forms - Claude

```
using QuizArcor.Models;

namespace QuizArcor.Services;

public static class CalculadoraPartida
{
    private static readonly string[] Faixas =
    {
        "Até 12 anos", "13 a 17 anos", "18 a 24 anos",
        "25 a 39 anos", "40 anos ou mais", "Prefiro não informar"
    };

    // Devolve a lista de erros (vazia = dados válidos)
    public static List<string> Validar(DadosPartida d)
    {
        var erros = new List<string>();

        if (string.IsNullOrWhiteSpace(d.Nickname))
            erros.Add("Nickname ausente.");
        if (d.FaixaEtaria < 1 || d.FaixaEtaria > 6)
            erros.Add("Faixa etária inválida.");
        if (d.AcertosFaceis < 0 || d.AcertosFaceis > d.QuestoesFaceis)
            erros.Add("Acertos nas fáceis inválidos.");
        if (d.AcertosMedias < 0 || d.AcertosMedias > d.QuestoesMedias)
            erros.Add("Acertos nas médias inválidos.");
        if (d.AcertosDificeis < 0 || d.AcertosDificeis > d.QuestoesDificeis)
            erros.Add("Acertos nas difíceis inválidos.");

        int total = d.QuestoesFaceis + d.QuestoesMedias + d.QuestoesDificeis;
        if (total < 1)
            erros.Add("A soma das questões dos três níveis deve ser pelo menos
        if (d.TempoPartida <= 0)
            erros.Add("Tempo da partida inválido.");
        if (d.DicasUsadas < 0 || d.DicasUsadas > total)
            erros.Add("Quantidade de dicas inválida.");

        return erros;
    }

    // Só chame depois de Validar() retornar lista vazia
    public static ResultadoPartida Calcular(DadosPartida d)
    {
        int total = d.QuestoesFaceis + d.QuestoesMedias + d.QuestoesDificeis;
        int acertos = d.AcertosFaceis + d.AcertosMedias + d.AcertosDificeis;
```

07/11/2025, 22:47 Projeto C# web Arcor com Windows Forms - Claude

```
using System.Net.Http.Json;
using QuizArcor.Models;

namespace QuizArcor.Services;

public class QuizApiService
{
    private static readonly HttpClient _http = new()
    {
        BaseAddress = new Uri("https://SEU-SERVIDOR.com/"), // troque pela URL
        Timeout = TimeSpan.FromSeconds(10)
    };

    // GET: busca as perguntas em JSON
    public async Task<List<Pergunta>> ObterPerguntasAsync()
    {
        var perguntas = await _http.GetFromJsonAsync<List<Pergunta>>("api/pergu
        return perguntas ?? new List<Pergunta>();
    }

    // POST: envia o resultado em JSON
    public async Task<bool> EnviarResultadoAsync(string jogador, int pontos)
    {
        var resposta = await _http.PostAsJsonAsync("api/resultados", new { joga
        return resposta.IsSuccessStatusCode;
    }
}
```

`GetFromJsonAsync` e `PostAsJsonAsync` já fazem a serialização e desserialização com `System.Text.Json`, sem precisar instalar pacote nenhum (.NET 6 ou superior).

3. Uso no formulário (Form1.cs)

07/11/2025, 22:47 Projeto C# web Arcor com Windows Forms - Claude

```
using QuizArcor.Models;
using QuizArcor.Services;

namespace QuizArcor;

public partial class Form1 : Form
{
    private readonly QuizApiService _api = new();
    private List<Pergunta> _perguntas = new();
    private int _indice, _pontos;

    public Form1() => InitializeComponent();

    private async void Form1_Load(object sender, EventArgs e)
    {
        try
        {
            _perguntas = await _api.ObterPerguntasAsync();
            MostrarPergunta();
        }
        catch (HttpRequestException ex)
        {
            MessageBox.Show($"Erro ao acessar o servidor: {ex.Message}");
        }
    }

    private void MostrarPergunta()
    {
        var p = _perguntas[_indice];
        lblEnunciado.Text = p.Enunciado;
        // preencha seus RadioButtons/botões com p.Opcoes
    }
}
```

Os controles `lblEnunciado` e os botões das opções você cria no Designer.

Pontos importantes para a entrega

- **Servidor externo:** ele precisa devolver JSON no formato do modelo. Se você ainda não tem um, dá para usar uma API pública de trivia ou subir um JSON simples (por exemplo, via GitHub Pages ou um mock como `mockapi.io`) só para demonstrar a chamada.
- Nunca trave a interface: use sempre `async`/`await`, como acima, para a tela não congelar durante a chamada.
- Trate erros: servidor fora do ar e timeout têm que mostrar uma mensagem, não derrubar o programa.

Trechos ou entregas afetadas	O Front-End juntamente com a parte de integrar todos as funções do jogo (Banco de dados do SQLite, etc) foi influenciado. Já havíamos fundamentado o código de BackEnd juntamente com o Professor Marcelo na primeira entrega da disciplina de Algoritmos e Lógica de Programação, contudo tivemos de complementar. Parte das entregas foram revisadas com a inteligência artificial também. Além da revisão das disciplinas das entregas 1 e 2, da complementação de partes do código, e geração de imagens, a IA também foi utilizada para descobrir como alterar o README.md e colocar imagens lá por exemplo. Vale a observação: Todos os arquivos foram revisados por todos os integrantes do grupo, bem como os códigos foram testados e reescritos. Sabendo disso, não foi usado prompt totalmente pronto onde o projeto completo seria disponibilizado com um "clique". Todas as pastas foram pensadas para atender a todos requisitos de negócio descritos na documentação do projeto, bem como para atender às exigências e expectativas da empresa convidada (e às expectativas dos integrantes), além de cumprir com os prazos estabelecidos. Diante disso, e apenas com base nisso, a utilização das ferramentas, sem intenção alguma de plágio ou de terceirizar o processo de desenvolvimento.

Declaro que o uso acima descrito está em conformidade com o GUIA PARA O USO RESPONSÁVEL DE INTELIGÊNCIA ARTIFICIAL NA FECAP e que o conteúdo final é de minha plena responsabilidade autoral.

07/11/2026

Data

Assinatura do(a) aluno(a)

Observação: a inclusão da Declaração de Uso de IA é obrigatória em atividades avaliativas oficiais (NI1, NI2, Prova Oficial) e trabalhos de conclusão de curso (Art. 6º, parágrafo único). A delegação de redação a ferramentas de IA sem declaração é equiparada a plágio (Art. 5º).